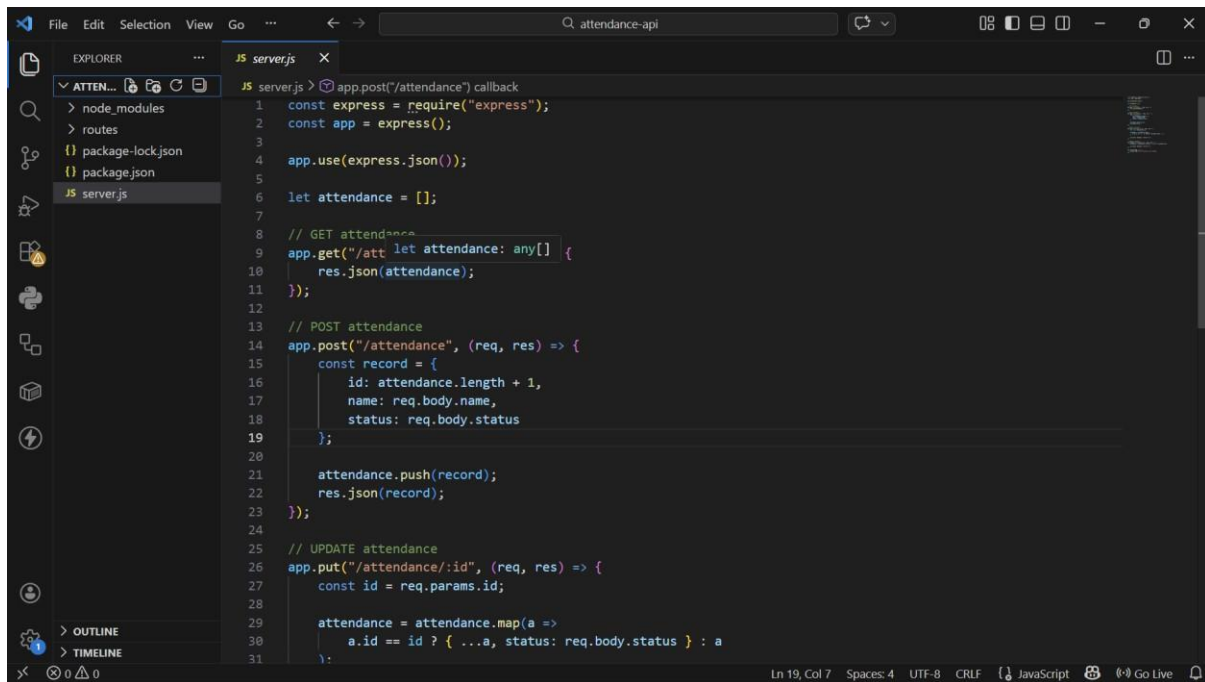


School of Computer Science and Artificial Intelligence

Lab Assignment-15.5

Program : B.Tech(CSE)
Specialization : AIML
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : VI
Academic Session: 2025-2026
Name of Student : k.sathwika
Enrollment No. : 2303A52096
Batch No. : 33
Date : 13/03/26

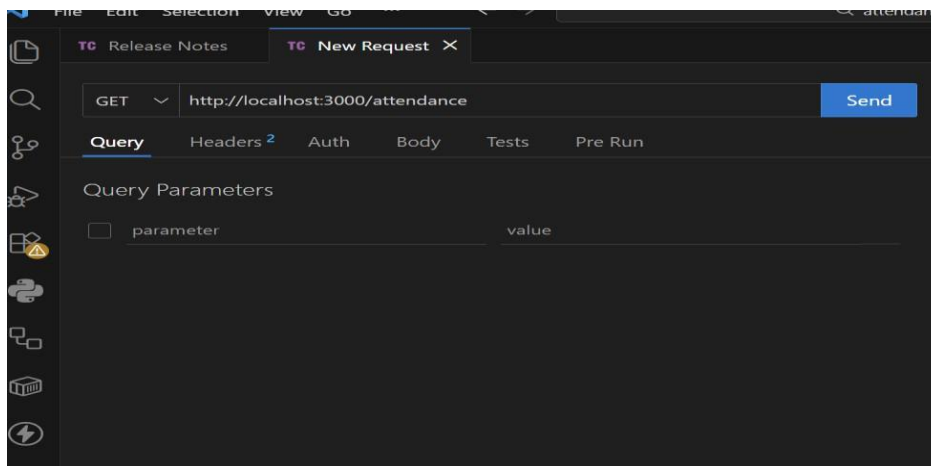
Task1–StudentAttendanceAPI



The screenshot shows a Visual Studio Code editor with a file named `server.js` open. The code is written in JavaScript and implements an Express.js server for a student attendance API. The file explorer on the left shows the project structure with files like `package-lock.json`, `package.json`, and `server.js`. The code in `server.js` includes the following logic:

```
1  const express = require("express");
2  const app = express();
3
4  app.use(express.json());
5
6  let attendance = [];
7
8  // GET attendance
9  app.get("/att let attendance: any[] {
10     res.json(attendance);
11 });
12
13 // POST attendance
14 app.post("/attendance", (req, res) => {
15     const record = {
16         id: attendance.length + 1,
17         name: req.body.name,
18         status: req.body.status
19     };
20
21     attendance.push(record);
22     res.json(record);
23 });
24
25 // UPDATE attendance
26 app.put("/attendance/:id", (req, res) => {
27     const id = req.params.id;
28
29     attendance = attendance.map(a =>
30         a.id == id ? { ...a, status: req.body.status } : a
31     );
```

GET/attendance→Viewattendancerecords



o **POST/attendance**→**Mark attendance**

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/attendance
- Body:** JSON format with the following content:

```
1 {
2   "name": "Sowmya",
3   "status": "Present"
4 }
```
- Status:** 200 OK
- Size:** 43 Bytes
- Time:** 9 ms
- Response:** JSON format with the following content:

```
1 {
2   "id": 1,
3   "name": "Sowmya",
4   "status": "Present"
5 }
```

o **PUT/attendance/{id}**→**Update attendance record**

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:3000/attendance/1
- Body:** JSON format with the following content:

```
1 {
2   "status": "Absent"
3 }
```
- Status:** 200 OK
- Size:** 21 Bytes
- Time:** 4 ms
- Response:** JSON format with the following content:

```
1 {
2   "message": "Updated"
3 }
```

o **DELETE/attendance/{id}→Removeattendanceentry**

The screenshot shows a REST client interface with a dark theme. At the top, there are tabs for "Release Notes" and "New Request". Below these, a dropdown menu is set to "DELETE" and the URL is "http://localhost:3000/attendance/1". A blue "Send" button is to the right. Below the URL bar, there are tabs for "Query", "Headers 2", "Auth", "Body 1", "Tests", and "Pre Run". The "Body 1" tab is selected, and within it, the "JSON" format is chosen. The request body is a JSON object:

```
{  "status": "Absent"}
```

. Below the body editor, the status bar shows "Status: 200 OK", "Size: 21 Bytes", and "Time: 4 ms". The "Response" tab is active, showing the response body:

```
{  "message": "Deleted"}
```

DELETE ⌵ http://localhost:3000/attendance/1 Send

Query Headers ² Auth **Body ¹** Tests Pre Run

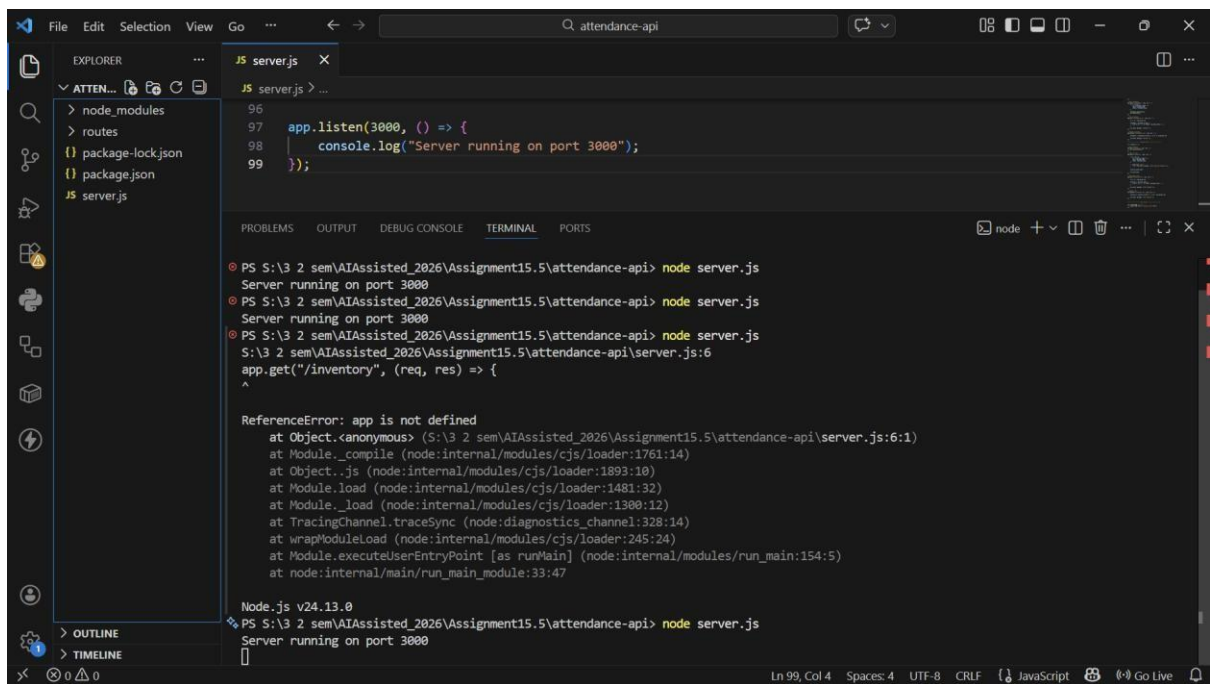
JSON XML Text Form Form-encode GraphQL

```
1 {
2   "status": "Absent"
3 }
```

Status: 200 OK Size: 21 Bytes Time: 4 ms Response

```
1 {
2   "message": "Deleted"
3 }
```

Task2-InventoryManagement API



The screenshot shows a Visual Studio Code editor with a file explorer on the left showing a project structure with files like package-lock.json, package.json, and server.js. The main editor displays the server.js file with the following code:

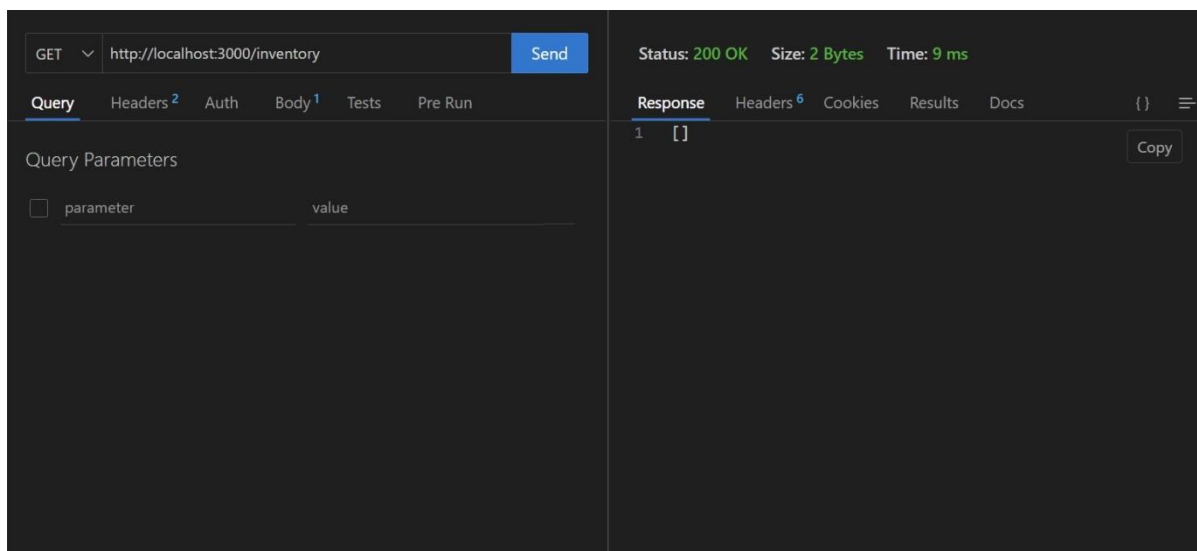
```
96
97 app.listen(3000, () => {
98   console.log("Server running on port 3000");
99 });
```

The terminal window at the bottom shows the command `node server.js` being executed, followed by the error message:

```
ReferenceError: app is not defined
    at Object.<anonymous> (S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6:1)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.13.0
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
```

GET/inventory→Listallitems

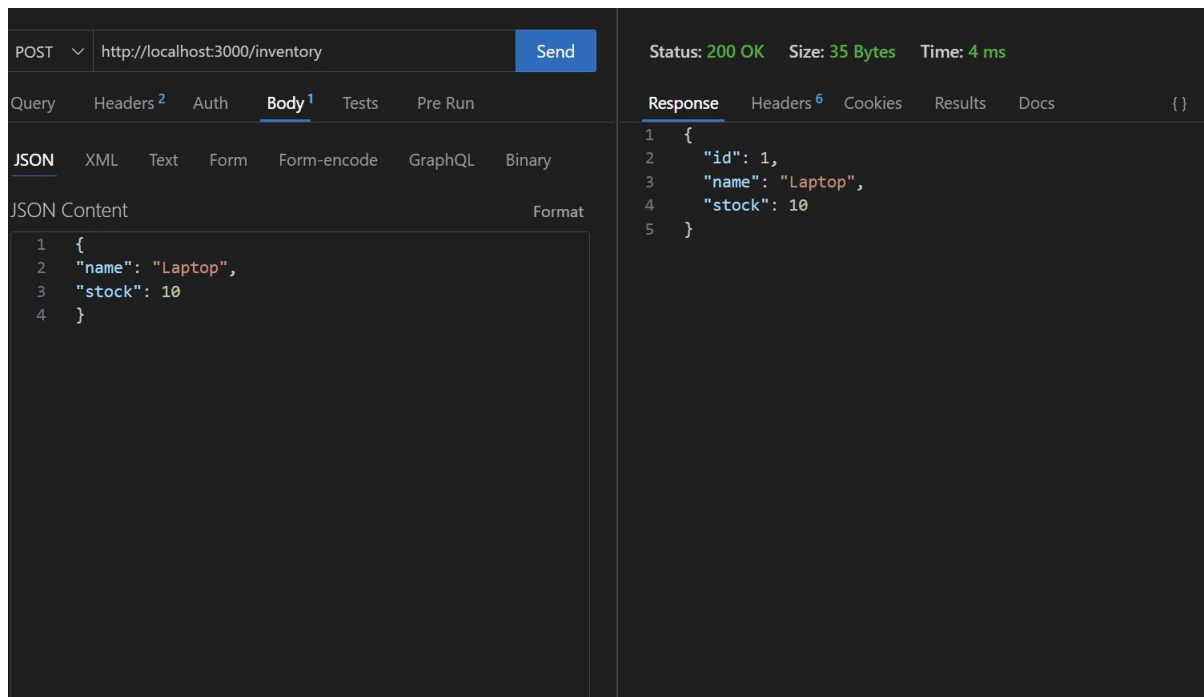


The screenshot shows a REST client interface with a GET request to `http://localhost:3000/inventory`. The response is a 200 OK status with a size of 2 Bytes and a time of 9 ms. The response body is an empty array `[]`.

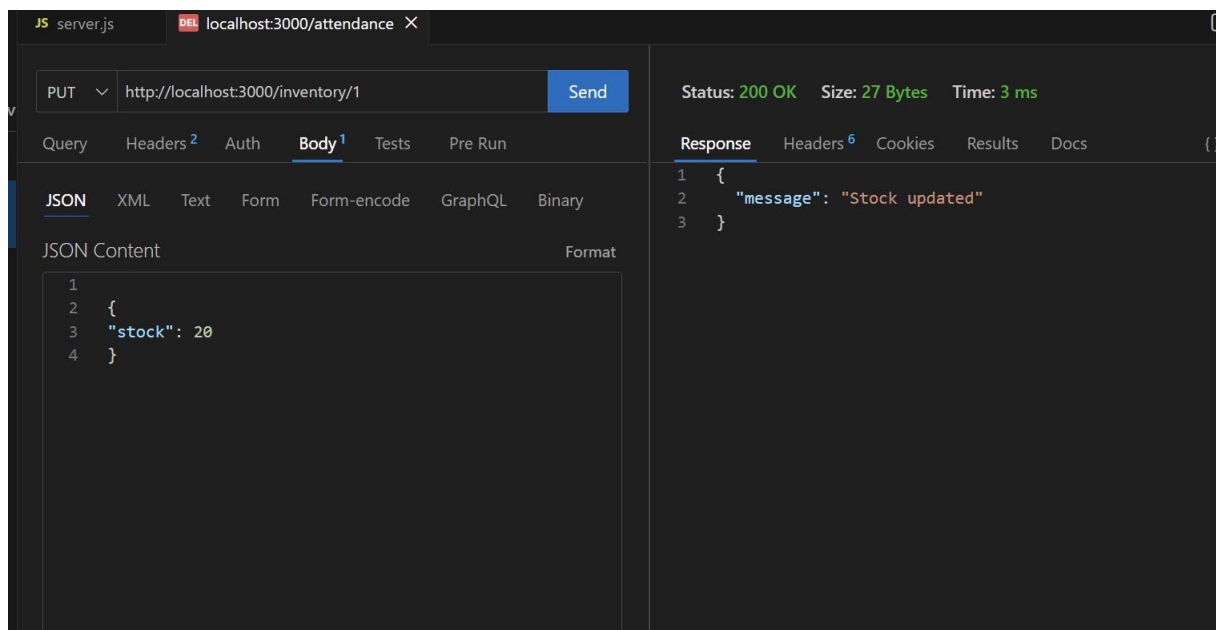
Query	Headers ²	Auth	Body ¹	Tests	Pre Run
Query Parameters					
☐ parameter			value		

Response	Headers ⁶	Cookies	Results	Docs
1	[]			

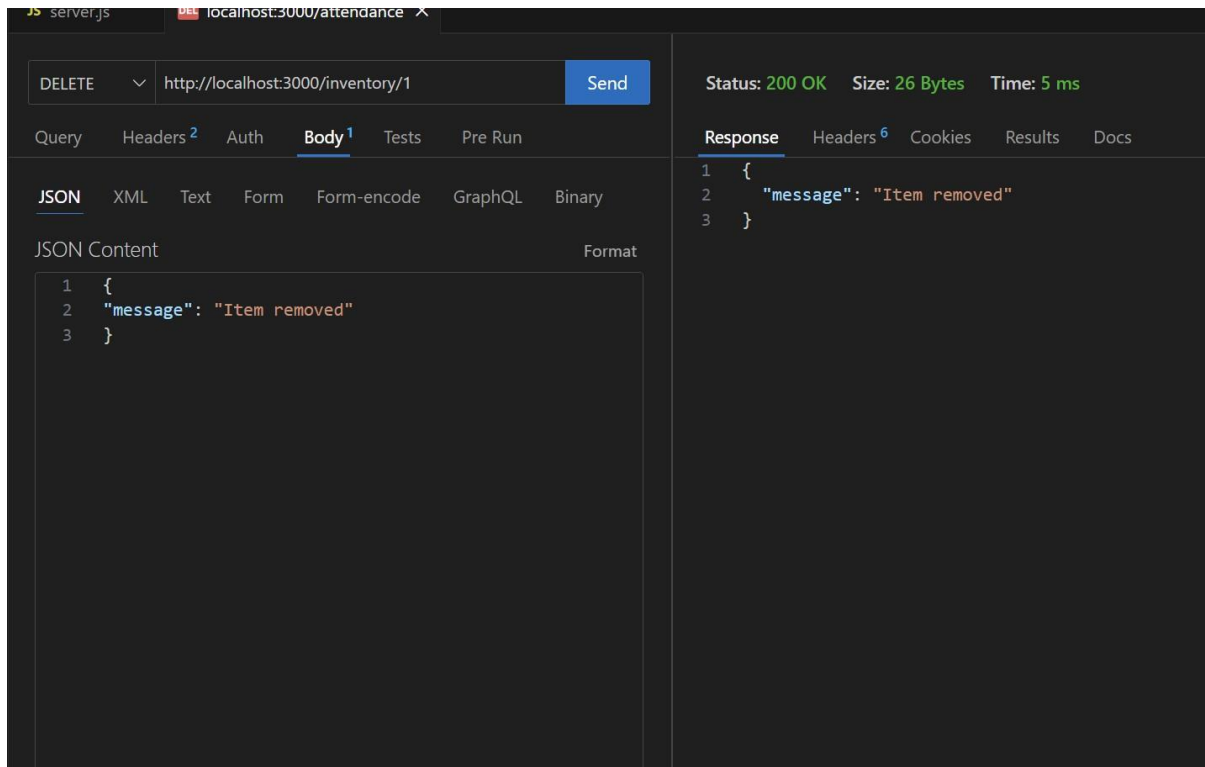
o POST/inventory→Addnewitem



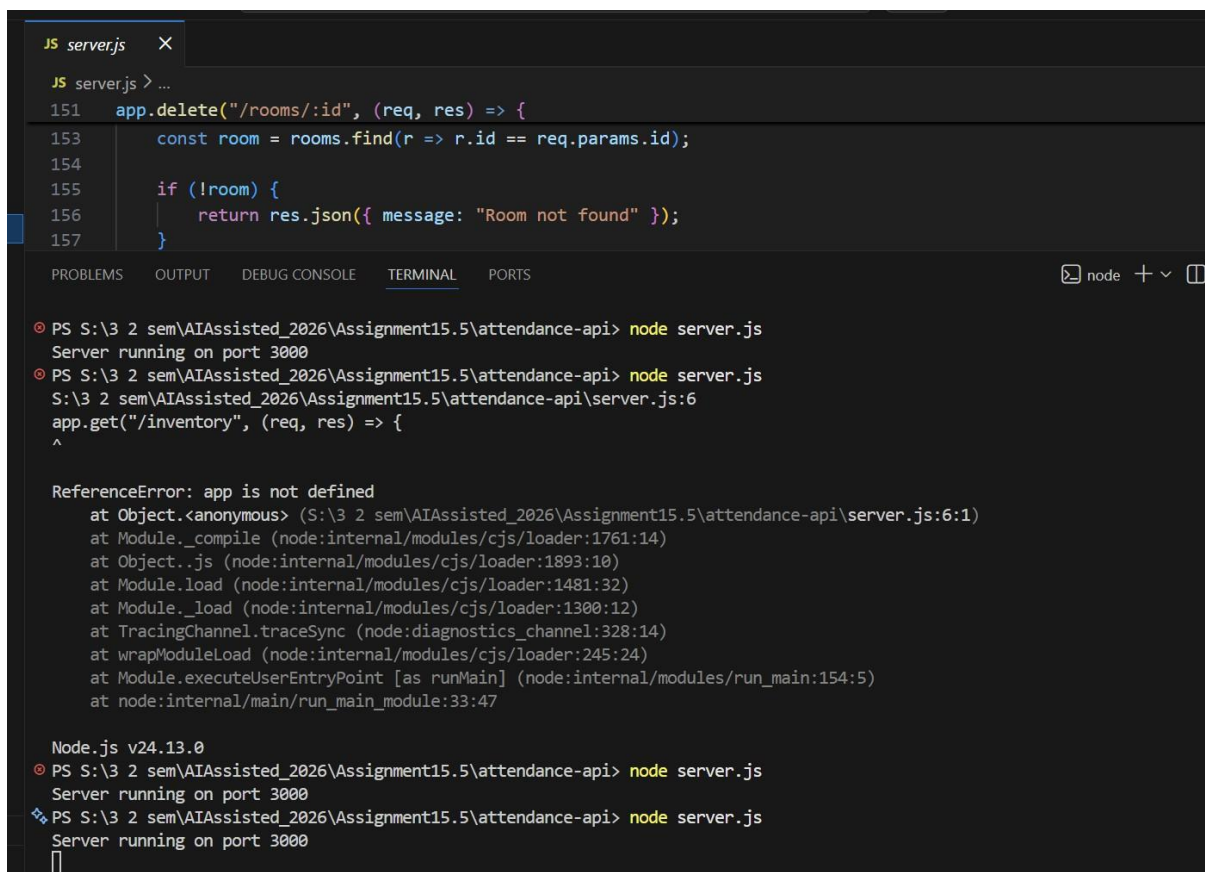
o **PUT/inventory/{id}→Update stock quantity**



o **DELETE/inventory/{id}→Remove an item**



Task3–HotelRoomBookingAPI



o GET/rooms→Viewavailablerooms

The screenshot shows a REST client interface with a tab labeled "TC New Request". The request method is "GET" and the URL is "http://localhost:3000/rooms". The "Send" button is visible. The "Query" tab is selected, showing "Query Parameters" with a table with columns "parameter" and "value". The "Response" tab is also visible, showing the status "200 OK", size "73 Bytes", and time "4 ms". The response body is a JSON array of three room objects:

```
1 [
2   {
3     "id": 1,
4     "booked": false
5   },
6   {
7     "id": 2,
8     "booked": false
9   },
10  {
11    "id": 3,
12    "booked": false
13  }
14 ]
```

o POST/rooms/book→Booka room

The screenshot shows a REST client interface with a tab labeled "TC New Request". The request method is "POST" and the URL is "http://localhost:3000/rooms/book". The "Send" button is visible. The "Body" tab is selected, showing "JSON Content" with a JSON object:

```
1 {
2   "id": 1
3 }
```

The "Response" tab is also visible, showing the status "200 OK", size "68 Bytes", and time "17 ms". The response body is a JSON object:

```
1 {
2   "message": "Room booked successfully",
3   "room": {
4     "id": 1,
5     "booked": true
6   }
7 }
```


o GET/rooms/{id}→Viewbooking details

The screenshot shows a REST client interface. The top bar displays the method **GET** and the URL `http://localhost:3000/rooms/1`, with a **Send** button. Below the top bar, the **Query** tab is selected, showing a table for query parameters with columns `parameter` and `value`. The right panel shows the **Response** tab with the following JSON data:

```
1 {
2   "id": 1,
3   "booked": true
4 }
```

Metadata at the top right of the response panel indicates: **Status: 200 OK**, **Size: 22 Bytes**, and **Time: 4 ms**.

o DELETE/rooms/{id}→Cancelbooking

The screenshot shows a REST client interface. The top bar displays the method **DELETE** and the URL `http://localhost:3000/rooms/1`, with a **Send** button. Below the top bar, the **Query** tab is selected, showing a table for query parameters with columns `parameter` and `value`. The right panel shows the **Response** tab with the following JSON data:

```
1 {
2   "message": "Booking cancelled"
3 }
```

Metadata at the top right of the response panel indicates: **Status: 200 OK**, **Size: 31 Bytes**, and **Time: 2 ms**.

Task4–OnlineVotingAPI

The screenshot shows a VS Code editor with a file explorer on the left. The file explorer shows a project named 'ATTENDANCE-API' with files: 'node_modules', 'routes', 'package-lock.json', 'package.json', and 'server.js'. The 'server.js' file is open in the editor. The code is as follows:

```
1 // Import express
2 const express = require("express");
3 const app = express();
4
5 app.use(express.json());
6
7 // Candidate data
8 let candidates = [
9   { id: 1, name: "Alice", votes: 0 },
10  { id: 2, name: "Bob", votes: 0 },
11  { id: 3, name: "Charlie", votes: 0 }
12 ];
13
14 // GET all candidates
15 app.get("/candidates", (req, res) => {
16   res.json(candidates);
17 });
18
19 // POST vote for candidate
20 app.post("/vote", (req, res) => {
21
22   const candidate = candidates.find(c => c.id == req.body.id);
23
24   if (!candidate) {
25     return res.json({ message: "Candidate not found" });
26   }
27
28   candidate.votes++;
29
30   res.json({
31     message: "Vote recorded"
```

o GET/candidates→Listcandidates

The screenshot shows a VS Code editor with a REST client tab. The request is a GET request to 'http://localhost:3000/candidates'. The status is '200 OK', size is '103 Bytes', and time is '8 ms'. The response is a JSON array of candidate objects.

Query Parameters

parameter	value
-----------	-------

Response

```
1 [
2   {
3     "id": 1,
4     "name": "Alice",
5     "votes": 0
6   },
7   {
8     "id": 2,
9     "name": "Bob",
10    "votes": 0
11  },
12  {
13    "id": 3,
14    "name": "Charlie",
15    "votes": 0
16  }
17 ]
```

o POST/vote→Castavote

POST ⌵ http://localhost:3000/vote Send

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id":1
3 }
```

Status: 200 OK Size: 73 Bytes Time: 9 ms

Response Headers ⁶ Cookies Results Docs

```
1 {
2   "message": "Vote recorded",
3   "candidate": {
4     "id": 1,
5     "name": "Alice",
6     "votes": 1
7   }
8 }
```

o GET/results→View voting results

GET ⌵ http://localhost:3000/results Send

Query Headers ² Auth **Body ¹** Tests Pre Run

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

Status: 200 OK Size: 158 Bytes Time: 3 ms

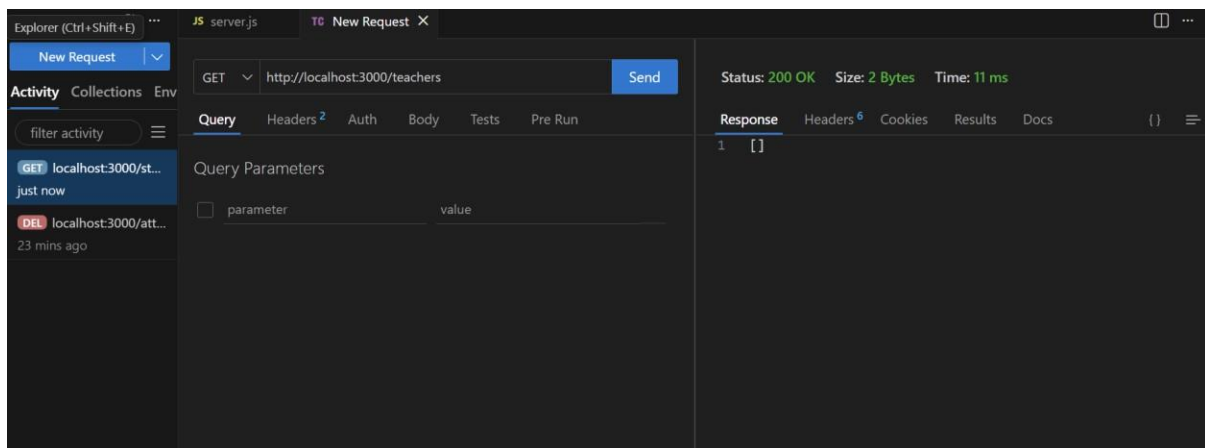
Response Headers ⁶ Cookies Results Docs {} Copy

```
1 {
2   "results": [
3     {
4       "id": 1,
5       "name": "Alice",
6       "votes": 1
7     },
8     {
9       "id": 2,
10      "name": "Bob",
11      "votes": 0
12     },
13     {
14       "id": 3,
15       "name": "Charlie",
16       "votes": 0
17     }
18   ],
19   "winner": {
20     "id": 1,
21     "name": "Alice",
22     "votes": 1
23   }
24 }
```

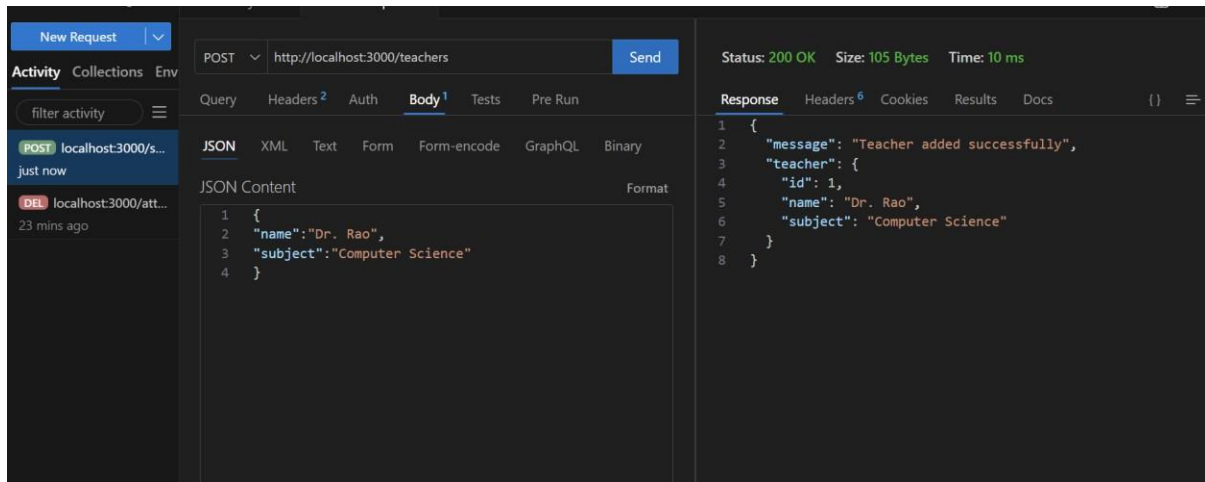
Task5–TeacherManagementAPI

```
JS server.js X TC New Request
JS server.js > app.put("/teachers/:id") callback > message
37 app.put("/teachers/:id", (req, res) => {
45   teacher.name = req.body.name || teacher.name;
46   teacher.subject = req.body.subject || teacher.subject;
47
48   res.json({
49     message: "Teacher updated",
50     teacher: teacher
51   });
52 });
53
54 // DELETE /teachers/:id → Remove teacher
55 app.delete("/teachers/:id", (req, res) => {
56
57   const teacher = teachers.find(t => t.id == req.params.id);
58
59   if (!teacher) {
60     return res.status(404).json({ message: "Teacher not found" });
61   }
62
63   teachers = teachers.filter(t => t.id != req.params.id);
64
65   res.json({ message: "Teacher removed successfully" });
66 });
67
68 // Start server
69 app.listen(3000, () => {
70   console.log("Server running on port 3000");
71 });
```

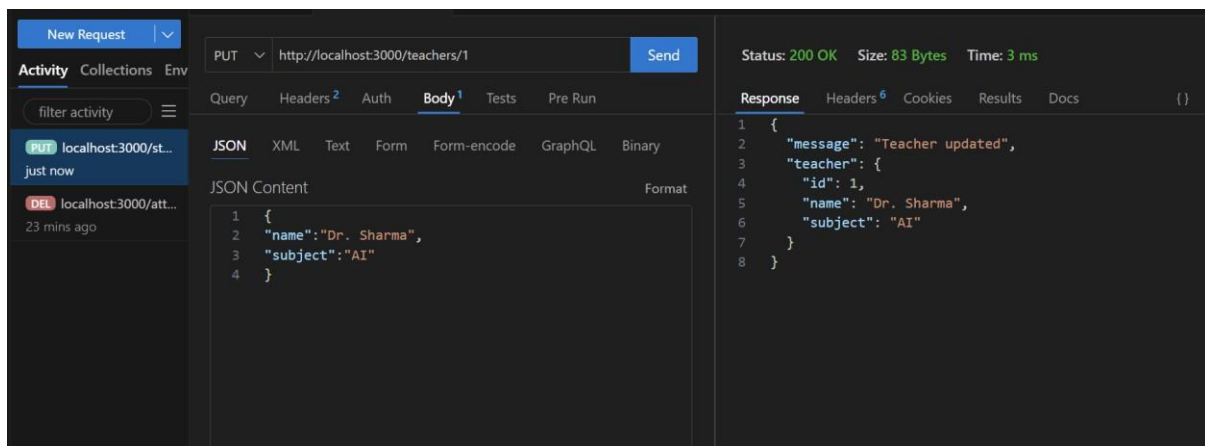
o GET/teachers→Listallteachers



o POST/teachers→Addanewteacher



o PUT/teachers/{id}→Update teacher details



o DELETE/teachers/{id}→Remove a teacher

